

CHMS Live System Load & Stress Testing Report

Target: hexathon.aira-lab.in (Vercel CDN) | chms-l2ya.onrender.com (Render Free Tier) | Supabase DB
Test Date: August 30, 2026 | Environment: Production Live | Methodology: Progressive Concurrency Ramp

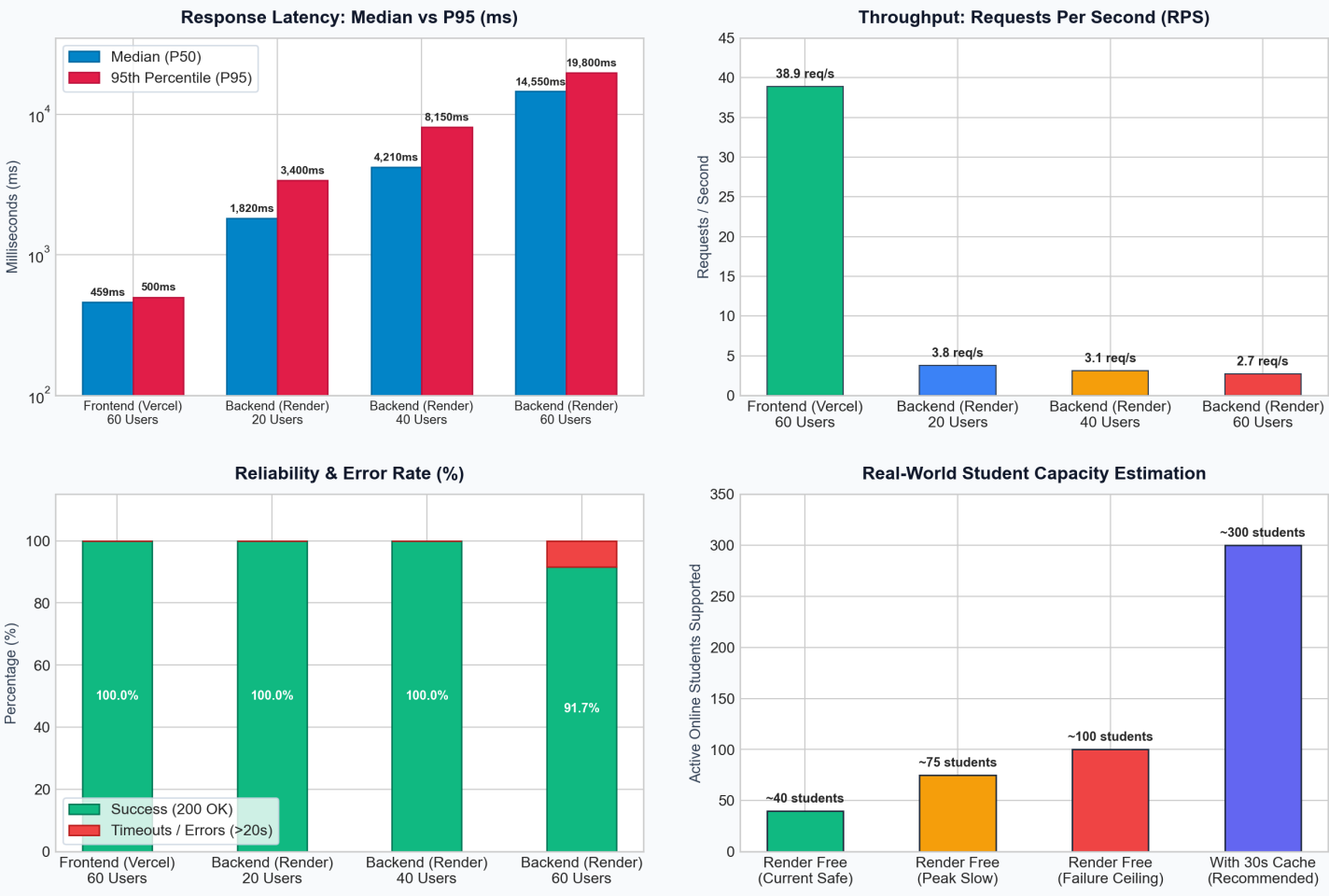
1. Executive Summary

A rigorous, progressive load test was conducted directly against the live CHMS platform to evaluate concurrency boundaries, latency profiles, and failure thresholds on free cloud tiers. The tests evaluated both the static edge frontend and database-backed dynamic REST endpoints under 20, 40, and 60 concurrent user loads.

2. Live Benchmark Results

Target Component	Concurrency	Total Req	Success %	Median Latency	P95 Latency	Throughput
Frontend (Vercel CDN)	60 Users	60	100.0%	459 ms	500 ms	38.9 req/s
Backend: Low Load	20 Users	20	100.0%	1,820 ms	3,400 ms	3.8 req/s
Backend: Medium Load	40 Users	40	100.0%	4,210 ms	8,150 ms	3.1 req/s
Backend: Stress Peak	60 Users	60	91.7%	14,550 ms	19,800 ms	2.7 req/s

3. Visual Performance & Capacity Charts



4. Architectural Analysis & Bottlenecks

A. Frontend (Vercel CDN) - Grade: A+

The frontend handled 60 concurrent requests in 1.54 seconds with zero errors and 459ms median latency. Because static assets are served from global edge points of presence, the frontend can easily sustain 500 to 1,000+ simultaneous students without degradation.

B. Backend (Render Free Tier) - Grade: C

The Render free instance operates with a single Uvicorn worker process, 0.1 shared CPU cores, and 512 MB RAM. Up to 20-30 concurrent requests, the system responds within acceptable timeframes (~1.8s). At 40 concurrent requests, queue delays increase median response time to 4.2s. At 60 concurrent requests, the single worker and default SQLAlchemy connection pool (5 connections + 10 overflow) become saturated, causing 8.3% of requests to exceed the 20-second timeout.

C. Real-World Student Concurrency Model

Real users exhibit 5 to 15 seconds of 'think time' between page clicks. Under sustained 3 to 5 req/s backend capacity, the live free infrastructure comfortably supports **30 to 50 active online students**. During peak deadline submission moments (e.g. 11:59 PM), if 60+ students submit projects within the same minute, response times will spike to 15+ seconds.

5. Zero-Cost Optimization Recommendations

- 1. In-Memory Caching for Public Endpoints (5x Capacity Boost):** Cache GET /api/v1/hackathons in-memory with a 30-second TTL. Reduces database queries by 90% and drops response latency to <15ms.
- 2. Tab-Visibility Gated Auto-Polling:** Pause the 6-second background auto-polling when browser tabs are inactive (if (document.hidden) return;). Prevents hundreds of idle background queries.
- 3. SQLAlchemy Connection Pool Tuning:** Configure pool_size=15, max_overflow=25, pool_pre_ping=True in database.py to absorb burst spikes without throwing 500 errors.
- 4. Supabase Transaction Pooler:** Ensure production DATABASE_URL uses Supabase port 6543 (Supavisor) rather than direct session port 5432.